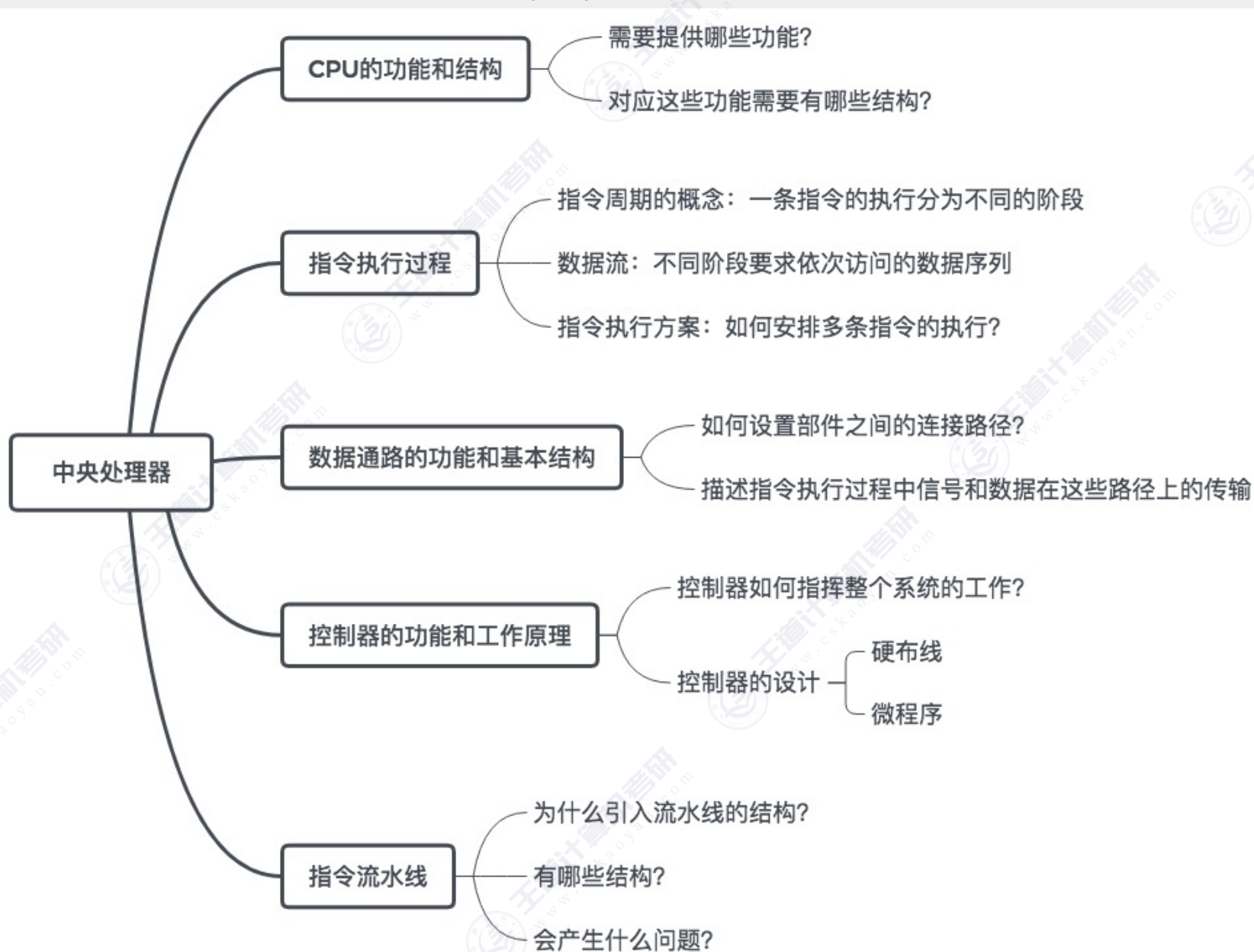


本节内容

控制器设计

硬布线控制器

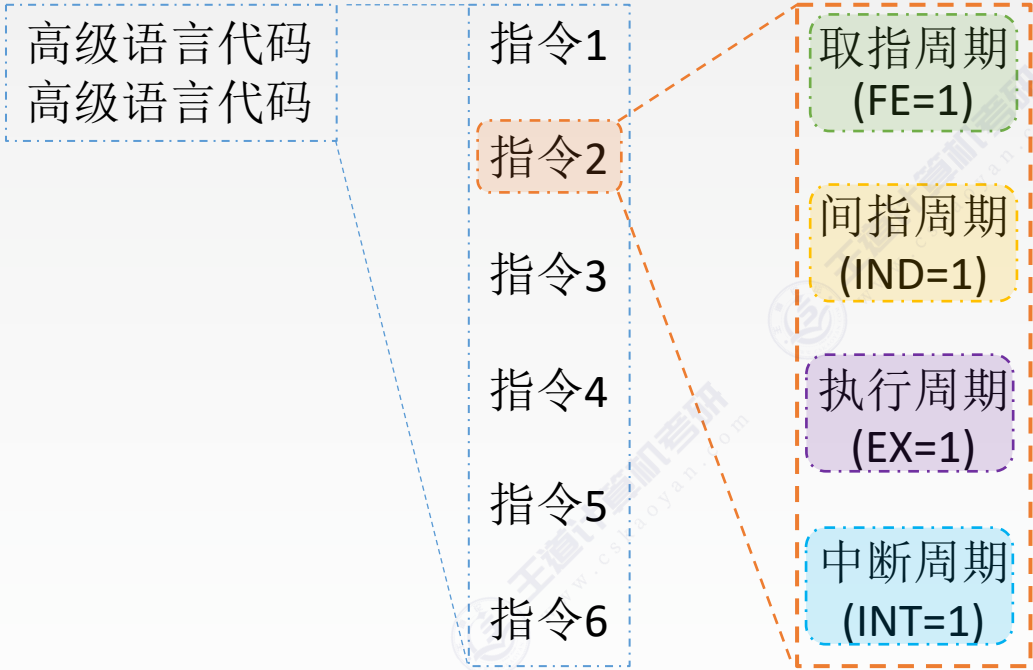
本章总览



根据 指令操作码、目前的机器周期、节拍信号、机器状态条件，即可确定现在这个节拍下应该发出哪些“微命令”

内容回顾

CU发出一个微命令，可完成对应微操作。
如：微命令1使得 PC_{out} 、 MAR_{in} 有效。
完成对应的微操作1 ($PC \rightarrow MAR$)



T_0 : 微操作1、微操作2
 T_1 : 微操作3
 T_2 : 微操作4

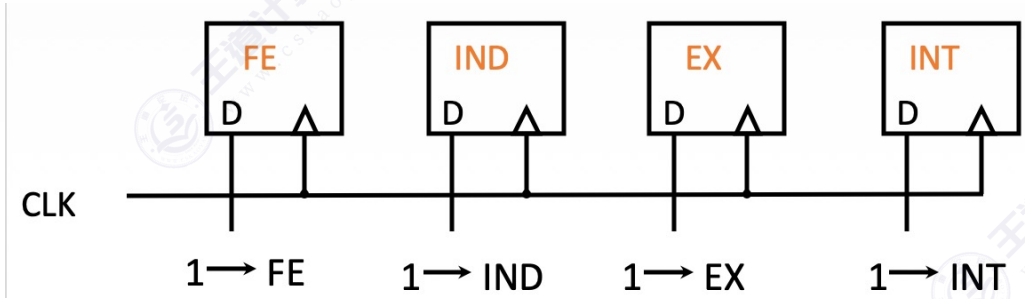
一个节拍内可以并行完成多个“相容的”微操作

T_0 : 微操作5、微操作2
 T_1 : 微操作6
 T_2 : 微操作7

同一个微操作可能在不同指令的不同阶段被使用

T_0 :
 T_1 : 微操作8
 T_2 : 微操作9、微操作6

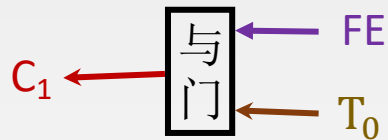
不同指令的执行周期所需节拍数各不相同。为了简化设计，选择定长的机器周期，以可能出现的最大节拍数为准（通常以访存所需节拍数作为参考）



T_0 :
 T_1 : 微操作10
 T_2 : 微操作11

若实际所需节拍数较少，可将微操作安排在机器周期末尾几个节拍上进行

所有指令的取指周期、 T_0 节拍下一定要完成 $(PC) \rightarrow MAR$ 。则可知 $C_1 = FE \cdot T_0$



Tips: 逻辑表达式是电路的数字化描述



每个时钟周期发出一个节拍信号 (循环发出)



T_0
 T_1
 $\vdots$
 T_m

硬布线控制器

指令寄存器IR

n位操作码

操作码译码器

0 1 ... $2^n - 1$

控制单元
CU

FE、IND、EX、INT (事实上这四个触发器集成在CU内部)

标志(来自执行单元的反馈信息)

可能来自运算器的PSW、ACC的符号位等; 也可能来自I/O设备、主存

C_0 C_1 ... C_k

每个输出的控制信号对应一个微命令, 也就是对应一个微操作

如: 要让 C_1 对应微操作 $(PC) \rightarrow MAR$, 则将其接到 PC_{out} 、 MAR_{in} 即可

根据 指令操作码、目前的机器周期、节拍信号、机器状态条件, 即可确定现在这个节拍下应该发出哪些“微命令”



用平静掩饰恐惧

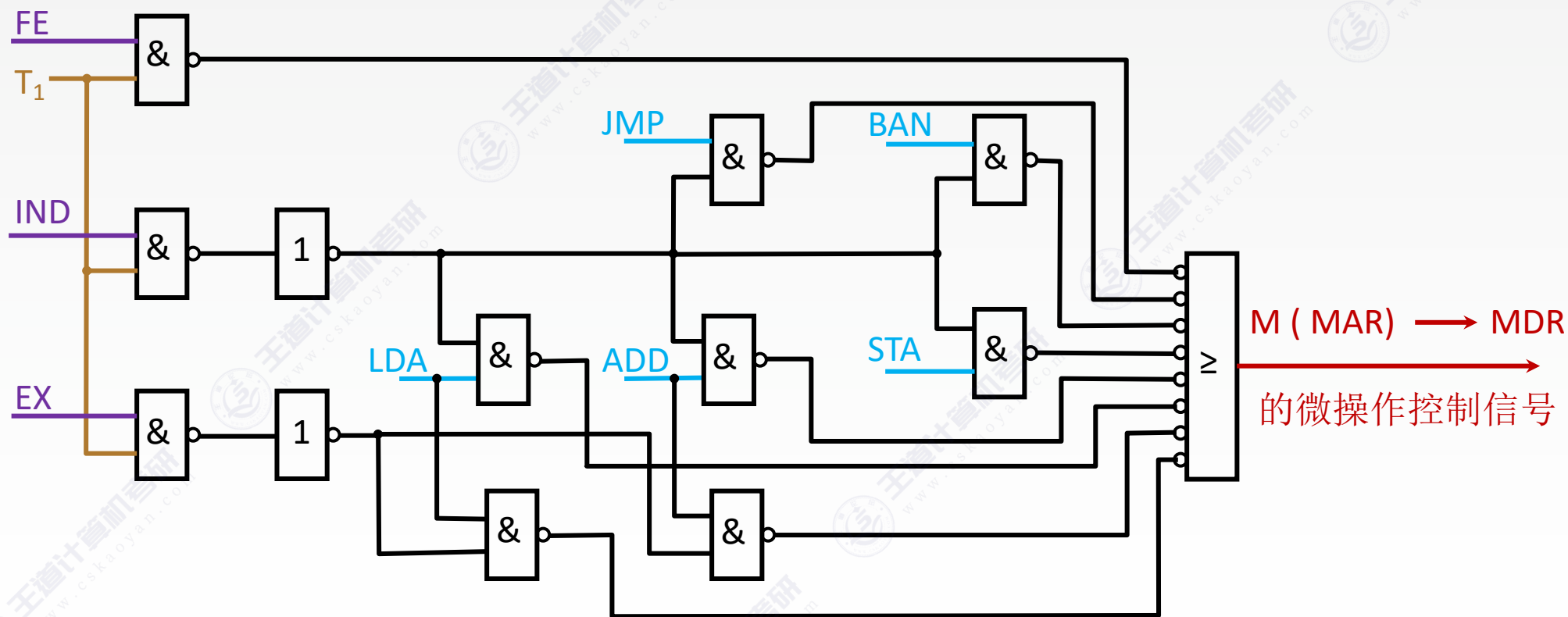
颤抖吧！感受恐惧！



很紧张对吧？

注：一般不考电路，莫慌~

$M(MAR) \rightarrow MDR$ 微操作命令的逻辑表达式：
 $FE \cdot T_1 + IND \cdot T_1(ADD + STA + LDA + JMP + BAN) + EX \cdot T_1(ADD + LDA)$



根据 指令操作码、目前的机器周期、节拍信号、机器状态条件，即可确定
现在这个节拍下应该发出哪些“微命令”

硬布线控制器的设计

设计步骤:

确定哪些指令在什么阶段、在什么条件下会使用到的微操作

1. 分析每个阶段的微操作序列（取值、间址、执行、中断 四个阶段）

2. 选择CPU的控制方式

采用定长机器周期还是不定长机器周期？每个机器周期安排几个节拍？

3. 安排微操作时序

如何用3个节拍完成整个机器周期内的所有微操作？

4. 电路设计

确定每个微操作命令的逻辑表达式，并用电路实现

假设采用同步控制方式（定长机器周期），一个机器周期内安排3个节拍。

安排，必须安排



注：中断周期内的微操作序列就不分析了，原理类似

分析每个阶段的微操作序列

取指周期（所有指令都一样）

$PC \rightarrow MAR$

$1 \rightarrow R$

$M(MAR) \rightarrow MDR$

$MDR \rightarrow IR$

$OP(IR) \rightarrow ID$

$(PC) + 1 \rightarrow PC$

注：ID 是指令译码器
Instruction Decoder

间址周期（所有指令都一样）

$Ad(IR) \rightarrow MAR$

$1 \rightarrow R$

$M(MAR) \rightarrow MDR$

$MDR \rightarrow Ad(IR)$

执行周期（各不相同）

CLA

clear ACC 指令
ACC清零

LDA X

取数指令，
把X所指内容
取到ACC

JMP X

无条件转移

BAN X

Branch ACC Negative

条件转移，当ACC为负时转移

$0 \rightarrow AC$

$Ad(IR) \rightarrow MAR$

$1 \rightarrow R$

$M(MAR) \rightarrow MDR$

$MDR \rightarrow AC$

$Ad(IR) \rightarrow PC$

负数符号位为1

$A_0 \bullet Ad(IR) + \overline{A_0} \bullet (PC) \rightarrow PC$

罗列出所有指令在各个阶段的微操作序列，就可以知道在什么情况下需要使用这个微操作

根据指令操作码、目前的机器周期、节拍信号、机器状态条件，即可确定现在这个节拍下应该发出哪些“微命令”

注：很多地方把
ACC简写为AC

安排微操作时序的原则

原则一 微操作的 先后顺序不得 随意 更改

原则二 被控对象不同 的微操作

尽量安排在 一个节拍 内完成

原则三 占用 时间较短 的微操作

尽量 安排在 一个节拍 内完成

并允许有先后顺序

安排微操作时序-取指周期



原则一 微操作的 先后顺序不得 随意 更改

(1) $PC \rightarrow MAR$

原则二 被控对象不同 的微操作

(2) $1 \rightarrow R$

存储器空闲即可

尽量安排在一个节拍 内完成

(3) $M(MAR) \rightarrow MDR$

在(1)之后

原则三 占用 时间较短 的微操作

(4) $MDR \rightarrow IR$

在(3)之后

尽量 安排在一个节拍 内完成

(5) $OP(IR) \rightarrow ID$

在(4)之后

并允许有先后顺序

(6) $(PC) + 1 \rightarrow PC$

在(1)之后

安排微操作时序-取指周期

原则一 微操作的 先后顺序不得 随意 更改

原则二 被控对象不同 的微操作

尽量安排在一个节拍 内完成

原则三 占用 时间较短 的微操作

尽量 安排在一个节拍 内完成

并允许有先后顺序

T_0 (1) $PC \rightarrow MAR$

T_0 (2) $1 \rightarrow R$

存储器空闲即可

T_1 (3) $M(MAR) \rightarrow MDR$

在(1)之后

T_1 (6) $(PC) + 1 \rightarrow PC$

在(1)之后

T_2 (4) $MDR \rightarrow IR$

在(3)之后

T_2 (5) $OP(IR) \rightarrow ID$

在(4)之后

两个微操作占用时间较短，根据原则三安排在一个节拍

$M(MAR) \rightarrow MDR$ 从主存取数据，用时较长，因此必须一个时钟周期才能保证微操作的完成

$MDR \rightarrow IR$ 是CPU内部寄存器的数据传送，速度很快，因此在一个时钟周期内可以紧接着完成 $OP(IR) \rightarrow ID$ 。
也就是可以一次同时发出两个微命令。

安排微操作时序-间址周期



原则一 微操作的 先后顺序不得 随意 更改

T_0 (1) $Ad(IR) \rightarrow MAR$

原则二 被控对象不同 的微操作

T_0 (2) $1 \rightarrow R$

尽量安排在一个节拍 内完成

T_1 (3) $M(MAR) \rightarrow MDR$

原则三 占用 时间较短 的微操作

T_2 (4) $MDR \rightarrow Ad(IR)$

尽量 安排在一个节拍 内完成

并允许有先后顺序

安排微操作时序-执行周期

原则一 微操作的 先后顺序不得 随意 更改

原则二 被控对象不同 的微操作

尽量安排在一个节拍 内完成

原则三 占用 时间较短 的微操作

尽量 安排在一个节拍 内完成

并允许有先后顺序

① CLA

T_0

clear

T_1

ACC清零

T_2

$0 \rightarrow AC$

② COM

T_0

complement

T_1

ACC取反

T_2

$\overline{AC} \rightarrow AC$

③ SHR

T_0

shift

T_1

算术右移

T_2

$L(AC) \rightarrow R(AC)$

T_2

$AC_0 \rightarrow AC_0$

④ CSL

T_0

cyclic shift

T_1

循环左移

T_2

$R(AC) \rightarrow L(AC), AC_0 \rightarrow AC_n$

⑤ STP

T_0

stop

T_1

停机

T_2

$0 \rightarrow G$

安排微操作时序-执行周期

(1) 非访存指令

① CLA	T_0	
clear	T_1	
ACC清零	T_2	$0 \rightarrow AC$
② COM	T_0	
complement	T_1	
ACC取反	T_2	$\overline{AC} \rightarrow AC$
③ SHR	T_0	
shift	T_1	
算术右移	T_2	$L(AC) \rightarrow R(AC)$
	T_2	$AC_0 \rightarrow AC_0$
④ CSL	T_0	
cyclic shift	T_1	
循环左移	T_2	$R(AC) \rightarrow L(AC), AC_0 \rightarrow AC_n$
⑤ STP	T_0	
stop	T_1	
停机	T_2	$0 \rightarrow G$

(2) 访存指令

⑥ ADD X	T_0	$Ad(IR) \rightarrow MAR, 1 \rightarrow R$
加法指令	T_1	$M(MAR) \rightarrow MDR$
隐含ACC	T_2	$(AC) + (MDR) \rightarrow AC$
⑦ STA X	T_0	$Ad(IR) \rightarrow MAR, 1 \rightarrow W$
存数指令	T_1	$AC \rightarrow MDR$
隐含ACC	T_2	$MDR \rightarrow M(MAR)$
⑧ LDA X	T_0	$Ad(IR) \rightarrow MAR, 1 \rightarrow R$
取数指令	T_1	$M(MAR) \rightarrow MDR$
隐含ACC	T_2	$MDR \rightarrow AC$

(3) 转移指令

⑨ JMP X	T_0	
jump	T_1	
无条件转移	T_2	$Ad(IR) \rightarrow PC$
⑩ BAN X	T_0	
Branch ACC	T_1	
Negative	T_2	$A_0 \cdot Ad(IR) + \overline{A_0} \cdot (PC) \rightarrow PC$
条件转移		

安排微操作时序-中断周期

原则一 微操作的 先后顺序不得 随意 更改

原则二 被控对象不同 的微操作

尽量安排在一个节拍 内完成

原则三 占用 时间较短 的微操作

尽量 安排在一个节拍 内完成

并允许有先后顺序

T_0 (1) $a \rightarrow \text{MAR}$

T_0 (2) $1 \rightarrow W$

存储器空闲即可

T_0 (3) $0 \rightarrow \text{EINT}$

硬件关中断

T_1 (4) $(PC) \rightarrow \text{MDR}$

内部数据通路空闲即可

T_2 (5) $\text{MDR} \rightarrow \text{M}(\text{MAR})$

在(3)之后

T_2 (6) 向量地址 $\rightarrow PC$

在(3)之后

这些操作由中断隐指令完成

注：中断隐指令不是一条指令，而是指一条指令的中断周期由硬件完成的一系列操作

中断周期的三个任务：

1. 保存断点
2. 形成中断服务程序的入口地址
3. 关中断

设计步骤：

1. 分析每个阶段的微操作序列
2. 选择CPU的控制方式
3. 安排微操作时序
4. 电路设计

组合逻辑设计

设计步骤:

1. 列出操作时间表

列出在取指、间址、执行、中断周期，T0、T1、T2 节拍内有可能用到的所有微操作

2. 写出微操作命令的最简表达式

3. 画出逻辑图

组合逻辑设计

设计步骤:

1. 列出操作时间表

非访存指令

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	SHR	CSL	STP	ADD	STA	LDA	JMP	BAN
FE 取指	T ₀		PC → MAR	1	1	1	1	1	1	1	1	1	1
			1 → R	1	1	1	1	1	1	1	1	1	1
	T ₁		M(MAR) → MDR	1	1	1	1	1	1	1	1	1	1
			(PC) + 1 → PC	1	1	1	1	1	1	1	1	1	1
	T ₂		MDR → IR	1	1	1	1	1	1	1	1	1	1
			OP(IR) → ID	1	1	1	1	1	1	1	1	1	1
		I	1 → IND						1	1	1	1	1
		$\bar{I}$	1 → EX	1	1	1	1	1	1	1	1	1	1

间址特征

组合逻辑设计

设计步骤:

1. 列出操作时间表

非访存指令

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	SHR	CSL	STP	ADD	STA	LDA	JMP	BAN
IND 间址	T ₀		Ad (IR) → MAR						1	1	1	1	1
			1 → R						1	1	1	1	1
	T ₁		M(MAR) → MDR						1	1	1	1	1
	T ₂		MDR → Ad (IR)						1	1	1	1	1
		$\overline{\text{IND}}$	1 → EX						1	1	1	1	1

间址周期标志

组合逻辑设计

设计步骤:

1. 列出操作时间表

2. 写出微操作命令的最简表达式

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP	BAN
EX 执行	T ₀		Ad (IR) → MAR			1	1	1		
			1 → R			1		1		
			1 → W				1			
	T ₁		M(MAR) → MDR			1		1		
			AC → MDR				1			
	T ₂		(AC)+(MDR) → AC			1				
			MDR → M(MAR)				1			
			MDR → AC					1		
			0 → AC	1						
			$\overline{AC} \rightarrow AC$		1					
			Ad(IR) → PC						1	
		A ₀	Ad(IR) → PC							1

微操作信号综合

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	SHR	CSL	STP	ADD	STA	LDA	JMP	BAN
FE 取指	T ₀		PC → MAR	1	1	1	1	1	1	1	1	1	1
			1 → R	1	1	1	1	1	1	1	1	1	1
	T ₁		M(MAR) → MDR	1	1	1	1	1	1	1	1	1	1
IND 间址	T ₁		M(MAR) → MDR						1	1	1	1	1
EX 执行	T ₁		1 → W							1			
			M(MAR) → MDR						1		1		

M(MAR) → MDR微操作命令的逻辑表达式:

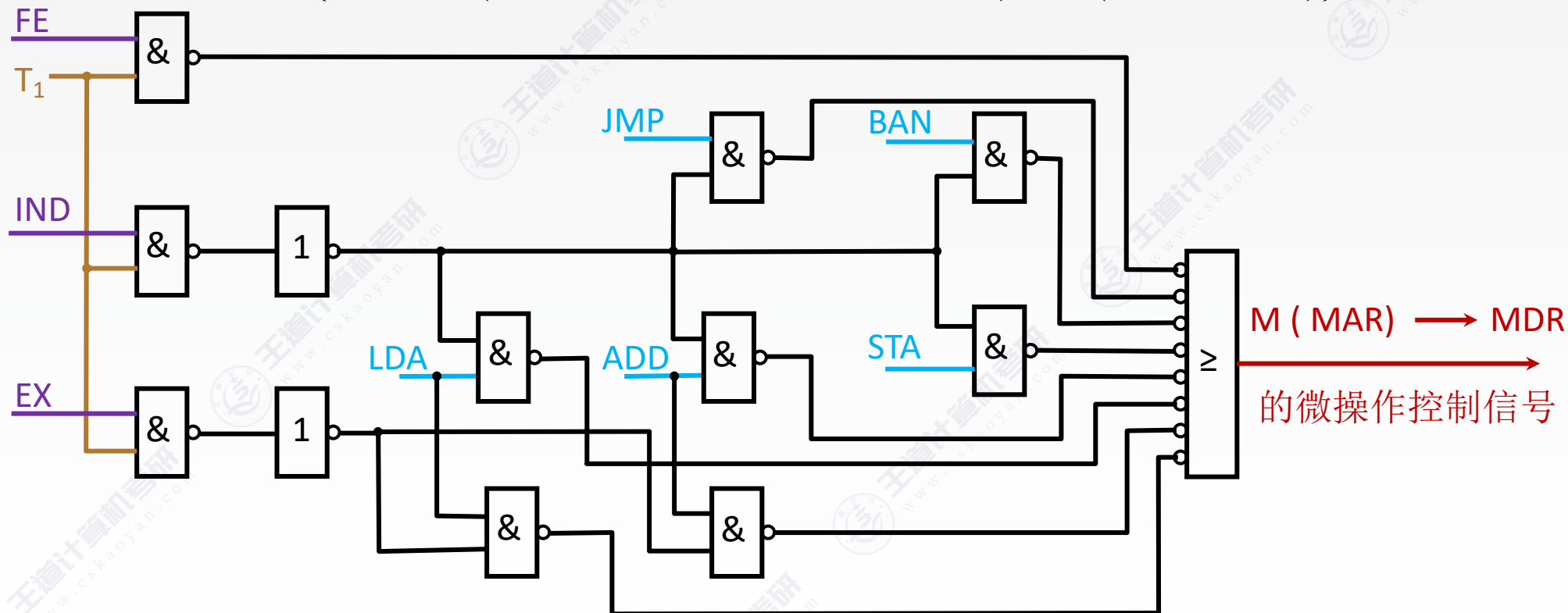
$$\begin{aligned}
 & \text{FE} \cdot T_1 + \text{IND} \cdot T_1 (\text{ADD} + \text{STA} + \text{LDA} + \text{JMP} + \text{BAN}) + \text{EX} \cdot T_1 (\text{ADD} + \text{LDA}) \\
 & = T_1 \{ \text{FE} + \text{IND} (\text{ADD} + \text{STA} + \text{LDA} + \text{JMP} + \text{BAN}) + \text{EX} (\text{ADD} + \text{LDA}) \}
 \end{aligned}$$

画出逻辑图

M (MAR) → MDR 微操作命令的逻辑表达式:

$$FE \cdot T_1 + IND \cdot T_1(ADD + STA + LDA + JMP + BAN) + EX \cdot T_1(ADD + LDA)$$

$$= T_1 \{ FE + IND(ADD + STA + LDA + JMP + BAN) + EX(ADD + LDA) \}$$



根据 指令操作码、目前的机器周期、节拍信号、机器状态条件，即可确定
现在这个节拍下应该发出哪些“微命令”

硬布线控制器的设计

设计步骤：

1. 分析每个阶段的微操作序列
2. 选择CPU的控制方式
3. 安排微操作时序
4. 电路设计
 - (1) 列出操作时间表
 - (2) 写出微操作命令的最简表达式
 - (3) 画出逻辑图

硬布线控制器的特点：

指令越多，设计和实现就越复杂，因此一般用于 RISC（精简指令集系统）

如果扩充一条新的指令，则控制器的设计就需要大改，因此扩充指令较困难。

由于使用纯硬件实现控制，因此执行速度很快。微操作控制信号由组合逻辑电路即时产生。



公众号：王道在线



b站：王道计算机教育

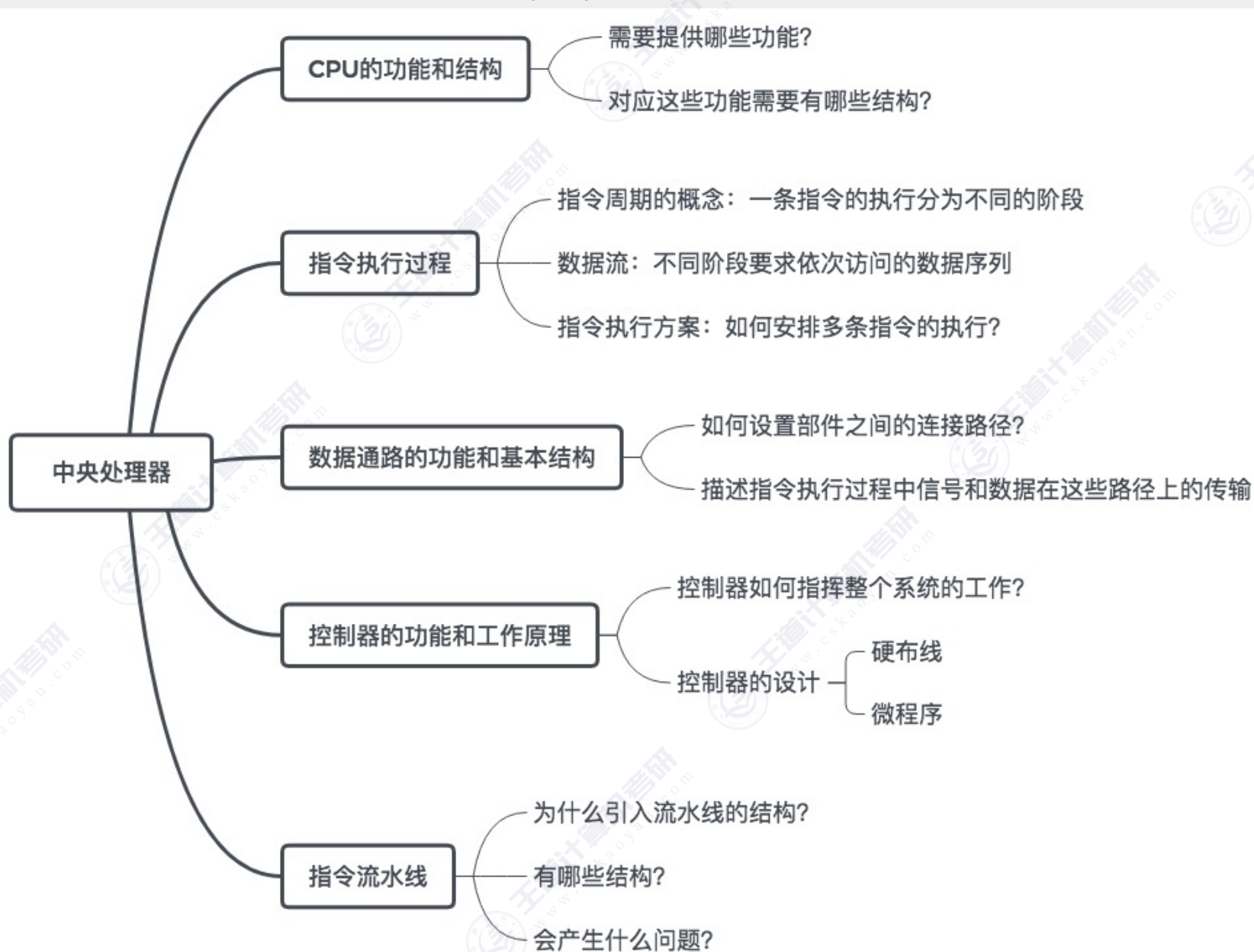


抖音：王道计算机考研

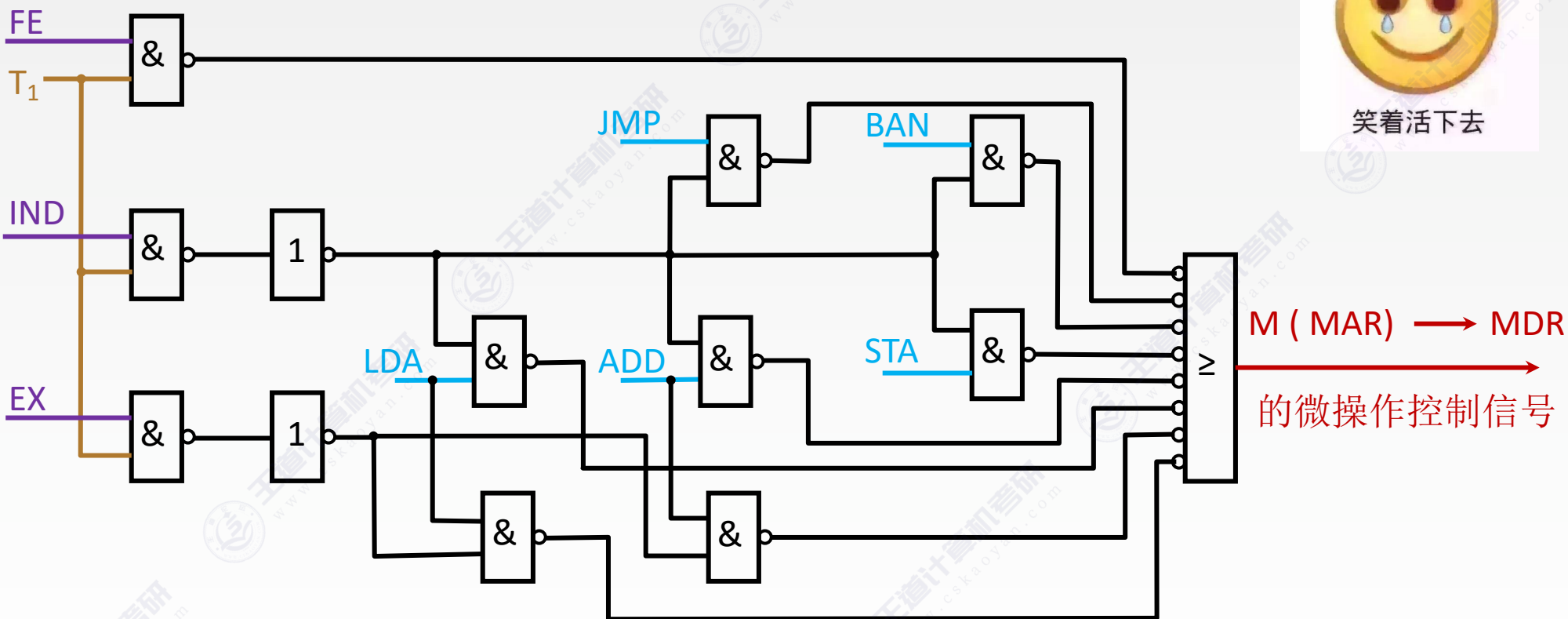
本节内容

微程序控制器 的基本原理

本章总览



硬布线控制器的设计思路



硬布线控制器：微操作控制信号由组合逻辑电路根据当前的指令码、状态和时序，即时产生

时序信息包含机
器周期、节拍

微程序控制器的设计思路

采用“存储程序”的思想，CPU 出厂前将所有指令的“微程序”存入“控制器存储器”中

高级语言代码
高级语言代码



指令1
指令2
指令3
指令4
指令5
指令6

取指周期
(FE=1)

间指周期
(IND=1)

执行周期
(EX=1)

中断周期
(INT=1)

T₀: 微操作1、微操作2
T₁: 微操作3
T₂: 微操作4

T₀: 微操作5、微操作2
T₁: 微操作6
T₂: 微操作7

T₀: 微操作7
T₁: 微操作8
T₂: 微操作9、微操作6

T₀: 微操作5、微操作2
T₁: 微操作10
T₂: 微操作11

微指令a: 完成微操作1、2
微指令b: 完成微操作3
微指令c: 完成微操作4

微指令d: 完成微操作5、2
微指令e: 完成微操作6
微指令f: 完成微操作7

微指令f: 完成微操作7
微指令g: 完成微操作8
微指令h: 完成微操作9、6

微指令d: 完成微操作5、2
微指令i: 完成微操作10
微指令j: 完成微操作11

微程序

程序：由指令序列组成

微程序：由微指令序列组成，每一种指令对应一个微程序

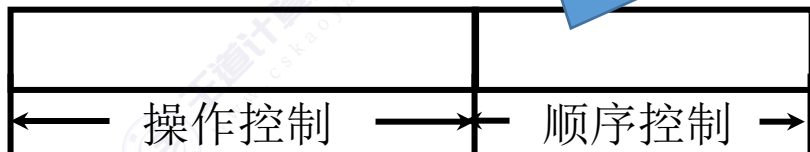
指令是对程序执行步骤的描述

微指令是对指令执行步骤的描述

指令是对微指令功能的“封装”

微指令基本格式

指明下一条微指令的地址



微命令与微操作一一对应
微指令中可能包含多个微命令

地址

P
K
指令1的微程序
指令2的微程序
微指令序列

微程序控制器的基本结构

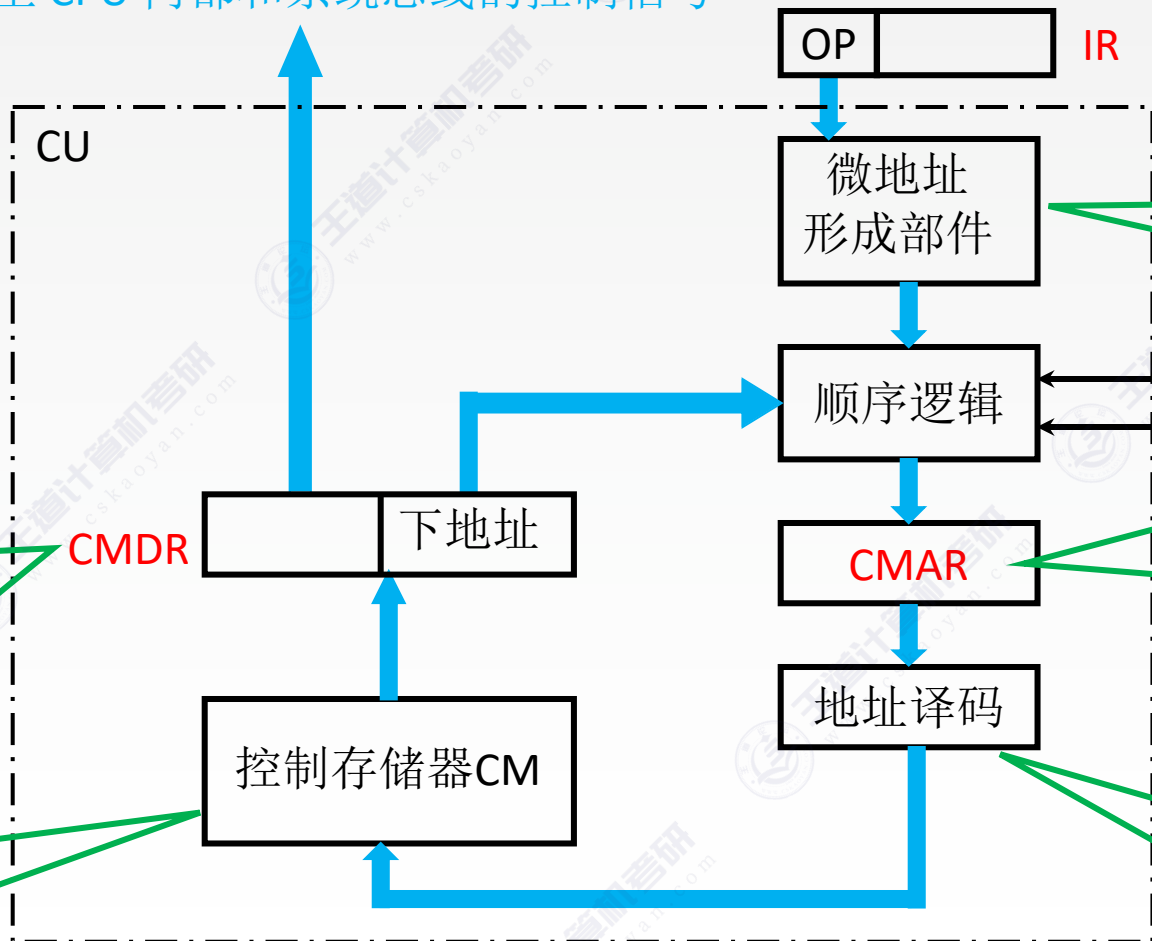
思考：所有指令的取指周期、间址周期、中断周期所对应的微指令序列都一样，是否可以共享使用？

至 CPU 内部和系统总线的控制信号

CM（按地址寻访）

别名：μIR，用于存放从CM中取出的微指令，它的位数同微指令字长相等。

用于存放各指令对应的微程序，控制存储器可用只读存储器ROM构成。



产生初始微地址和后继微地址，以保证微指令的连续执行。

标志
CLK

别名：μPC，微地址寄存器，接收微地址形成部件送来的微地址，为在CM中读取微指令作准备。

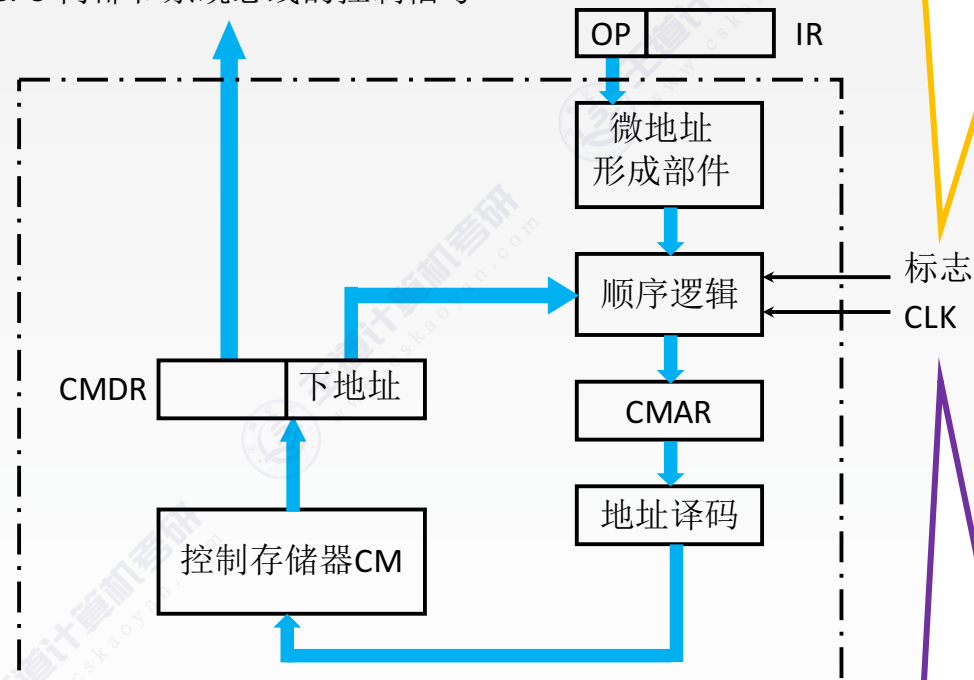
将地址码转化为存储单元控制信号。

微指令
存放地
址

微程序控制器的工作原理

微指令操作码	下地址	
	1	取指周期 微程序
	2	
	3	
⋮		间址周期 微程序
转执行周期		
⋮		
转取指周期		
⋮		中断周期 微程序
转取指周期		
⋮		
⋮		
	14	对应 LDA 指令的 执行周期微程序
	15	
	0	
	17	对应 STA 指令的 执行周期微程序
	18	
	0	
⋮		

至 CPU 内部和系统总线的控制信号



根据指令地址
码的寻址特征
位判断是否要
跳过间址周期

标志
CLK

根据中断信号
判断是否进入
中断周期

取数指令 LDA X

取指周期: #0、#1、#2

间址周期: #3、#4 #7

执行周期: #13、#14、#15

中断周期: #8、#9 #12

微指令
存放地
址

微程序控制器的工作原理

微指令操作码	下地址
	1
	2
	3
⋮	
转执行周期	
⋮	
转取指周期	
	14
	15
	0
	17
	18
	0
⋮	

取指周期 微程序

间址周期 微程序

中断周期 微程序

对应 LDA 指令的
执行周期微程序

对应 STA 指令的
执行周期微程序

通常是公用的，故如果某指令系统中有 n 条机器指令，则CM中微程序的个数至少是 $n+1$ 个

一些早期的CPU、物联网设备的CPU可以不提供间接寻址和中断功能，因此这类CPU可以不包含间址周期、中断周期的微程序段

Tips: 物理上，取指周期、执行周期看起来像是两个微程序，但逻辑上应该把它们看作一个整体。因此，“一条指令对应一个微程序”的说法是正确的

微程序控制器

CU 的结构

- 微地址形成部件
 - 微地址即微指令在 CM 中的存放地址
 - 通过指令操作码形成对应微程序的第一条微指令的存放地址
- 顺序逻辑
 - 根据某些机器标志和时序信息确定下一条微指令的存放地址
- CMAR (μPC)
 - 指明接下来要执行的微指令的存放地址
- 地址译码器
 - 将 CMAR 内的地址信息译码为电信号, 控制 CM 读出微指令
- 控制存储器 CM
 - 存放所有机器指令对应的微程序 (微指令序列)
 - 用 ROM 实现, 按地址寻访。通常在 CPU 出厂时就把所有微程序写入
- CMDR (μIR)
 - 微指令寄存器, 用于存放当前要执行的微指令。CM(μPC) $\rightarrow \mu IR$

工作原理

- 指令周期 = 取值周期 $\rightarrow$ 间址周期 $\rightarrow$ 执行周期 $\rightarrow$ 中断周期。其中间址、中断周期可有可无
- 处理取指周期、间址周期、中断周期的微指令序列通常是公用的。执行周期的微指令序列各不相同
- 取指周期的微指令序列固定从 #0 开始存放。执行周期的微指令序列的存放根据指令操作码确定

概念对比

- 程序 vs 微程序; 指令 vs 微指令; 主存储器 vs 控制器存储器 (CM); MAR vs CMAR; MDR vs CMDR; PC vs μPC ; IR vs μIR
- 微命令、微操作、微指令、微程序 之间的关系
- 指令周期: 从主存取出并执行一条机器指令所需的时间
- 微周期 (微指令周期): 从控制器存储器取出一条微指令并执行相应微操作所需的时间



公众号：王道在线



b站：王道计算机教育



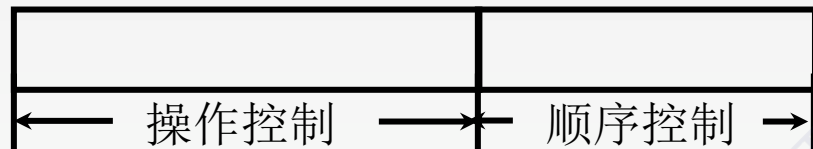
抖音：王道计算机考研

本节内容

微指令的设计

微程序控制器的工作原理

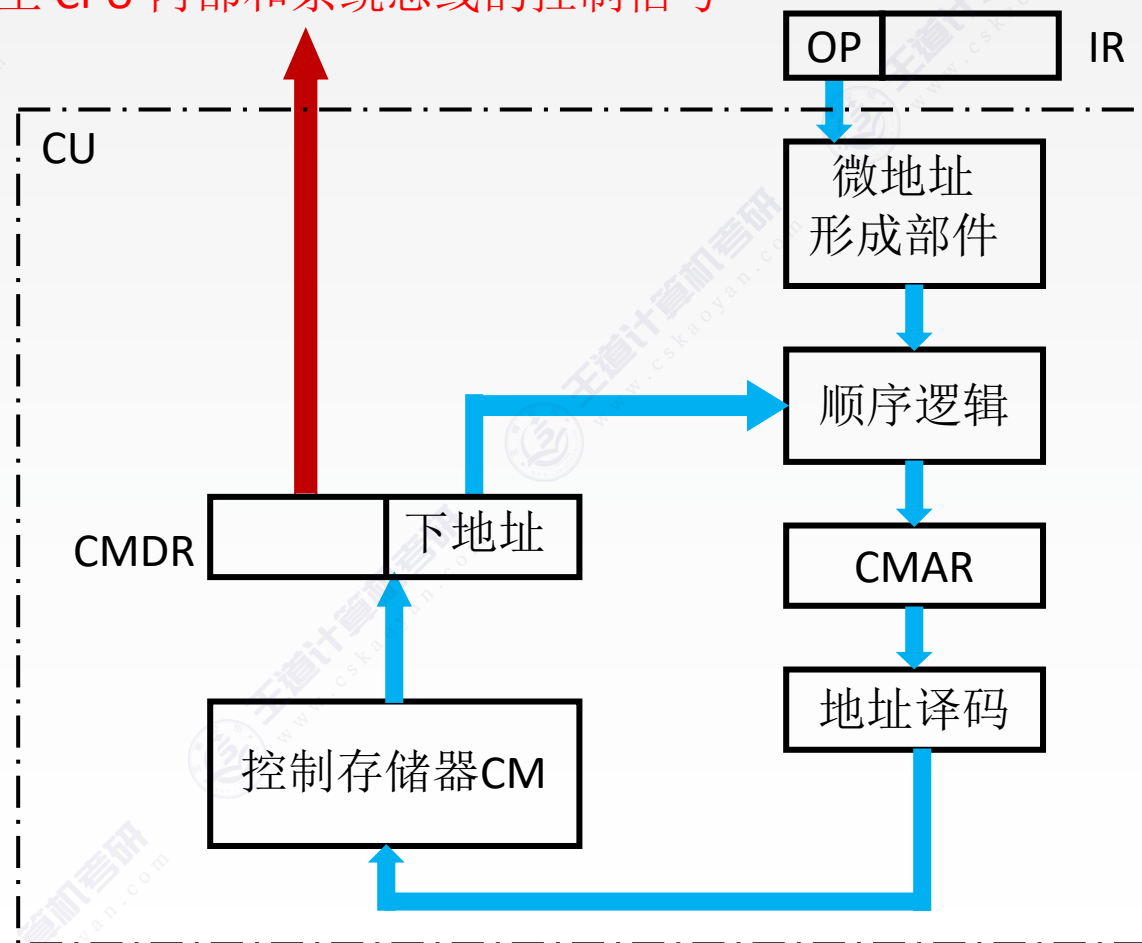
微指令基本格式



微指令的具体格式应该怎么设计？
如何根据微指令发出相应的微命令？

微命令与微操作一一对应，一个微命令对应一根输出线
有的微命令可以并行执行，因此一条微指令可以包含多个微命令

至 CPU 内部和系统总线的控制信号



微指令的格式

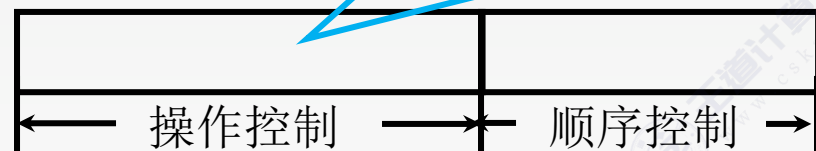
相容性微命令：可以并行完成的微命令。

互斥性微命令：不允许并行完成的微命令。

1. 水平型微指令 一条微指令能定义多个可并行的微命令。

基本格式

如何表示一系列控制信号？



优点：微程序短，执行速度快；

缺点：微指令长，编写微程序较麻烦。

微指令1
微指令2
微指令3

水平型（胖胖的）

2. 垂直型微指令 一条微指令只能定义一个微命令，由微操作码字段规定具体功能

基本格式



优点：微指令短、简单、规整，便于编写微程序；

缺点：微程序长，执行速度慢，工作效率低。

微指令0
微指令1
微指令2
微指令3
微指令4
微指令5
微指令6

垂直型
（瘦瘦的）

3. 混合型微指令 在垂直型的基础上增加一些不太复杂的并行操作。

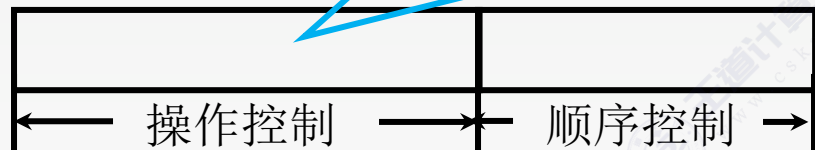
微指令较短，仍便于编写；微程序也不长，执行速度加快。

微指令的编码方式

1. 水平型微指令 一条微指令能定义多个可并行的微命令。

基本格式

如何表示一系列控制信号？



优点：微程序短，执行速度快；

缺点：微指令长，编写微程序较麻烦。

微指令的编码方式又称为微指令的控制方式，它是指如何对微指令的控制字段进行编码，以形成控制信号。编码的目标是在保证速度的情况下，尽量缩短微指令字长。

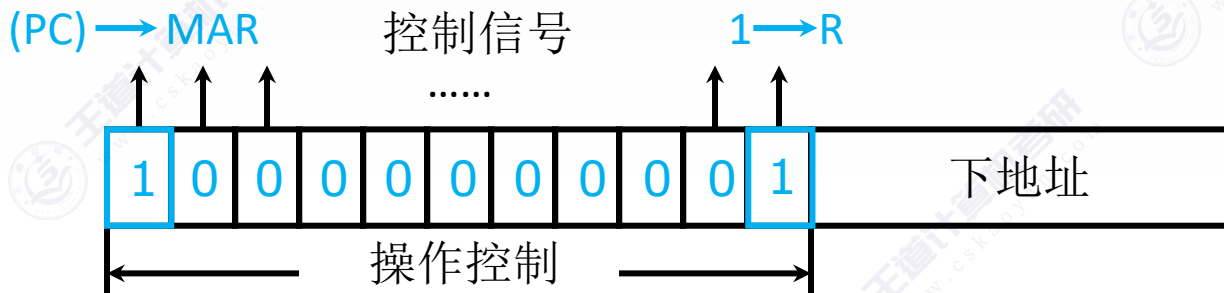
(1) 直接编码（直接控制）方式

在微指令的操作控制字段中，**每一位代表一个微操作命令**

某位为“1”表示该控制信号有效

优点：简单、直观，执行速度快，操作并行性好。

缺点：微指令字长过长， n 个微命令就要求微指令的操作字段有 n 位，造成控存容量极大。



微指令的编码方式

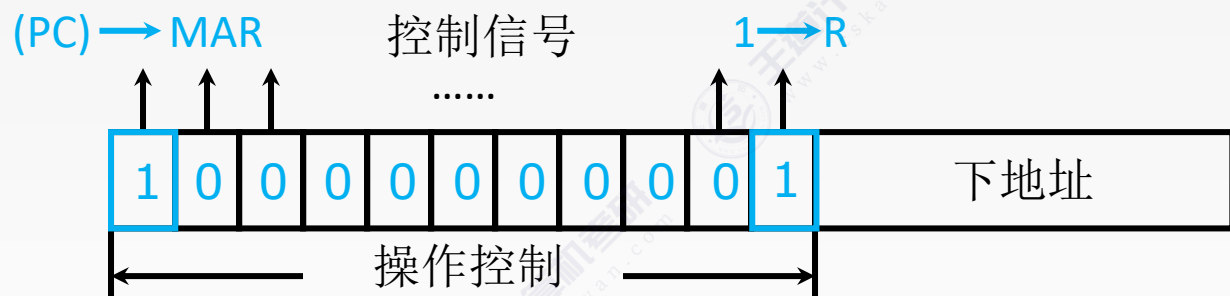
(1) 直接编码（直接控制）方式

在微指令的操作控制字段中，**每一位代表一个微操作命令**

某位为“**1**”表示该控制信号有效

优点：简单、直观，执行速度快，操作并行性好。

缺点：微指令字长过长， n 个微命令就要求微指令的操作字段有 n 位，造成控存容量极大。

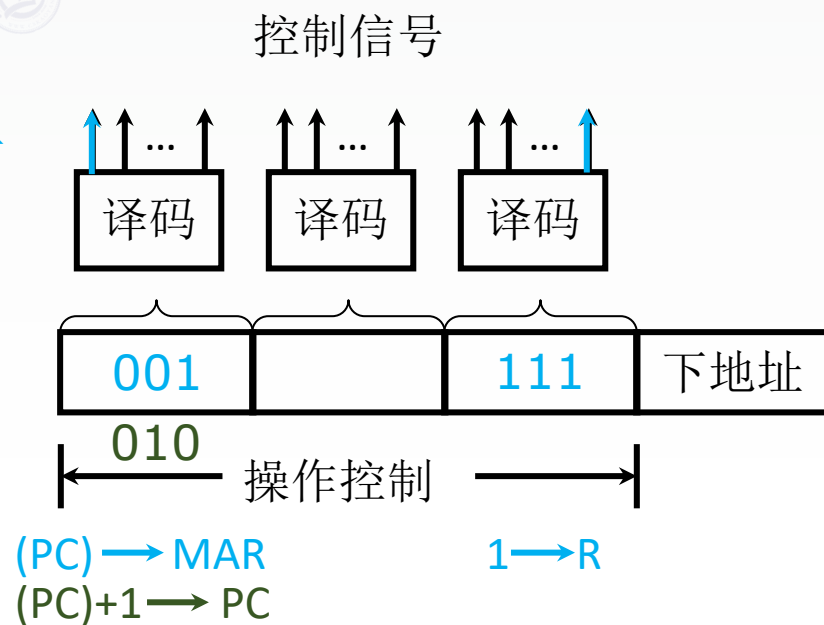


(2) 字段直接编码方式

将微指令的控制字段分成若干“段”，**每段经译码后发出控制信号**

微命令字段分段的原则：

- ① **互斥性**微命令分在**同一段内**，**相容性**微命令分在**不同段内**。
- ② **每个小段**中包含的**信息位不能太多**，否则将增加译码线路的复杂性和译码时间。
- ③ 一般**每个小段还要留出一个状态**，表示本字段不发出任何微命令。因此，当某字段的长度为3位时，最多只能表示7个互斥的微命令，**通常用000表示不操作**。

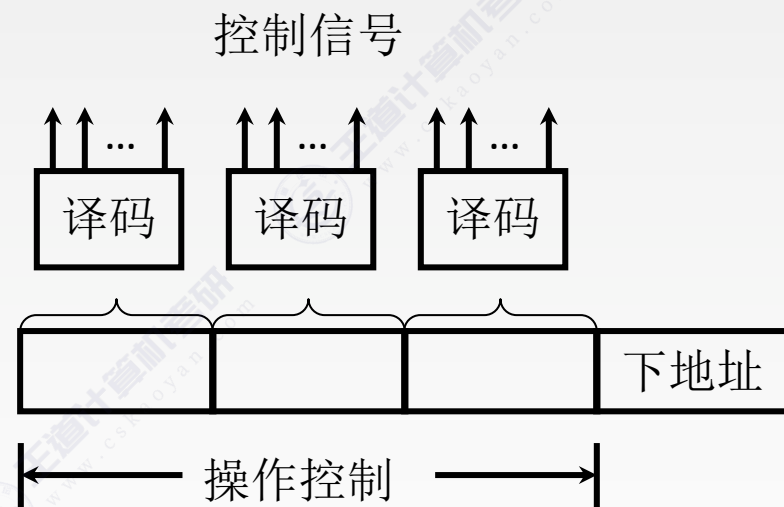


例题：字段直接编码方式

(2) 字段直接编码方式

将微指令的控制字段分成若干“段”，每段经译码后发出控制信号
微命令字段分段的原则：

- ① 互斥性微命令分在同一段内，相容性微命令分在不同段内。
- ② 每个小段中包含的信息位不能太多，否则将增加译码线路的复杂性和译码时间。
- ③ 一般每个小段还要留出一个状态，表示本字段不发出任何微命令。因此，当某字段的长度为3位时，最多只能表示7个互斥的微命令，通常用000表示不操作。



某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用**字段直接编码法**，共有33个微命令，构成5个互斥类，分别包含7、3、12、5和6个微命令，则操作控制字段至少有多少位？

第1个互斥类有7个微命令，要留出1个状态表示不操作，所以需要表示8种不同的状态，故需要3个二进制位。

以此类推，后面4个互斥类各需要表示4、13、6、7种不同的状态，分别对应2、4、3、3个二进制位。

故操作控制字段的总位数为

$$3+2+4+3+3 = 15 \text{ 位}$$

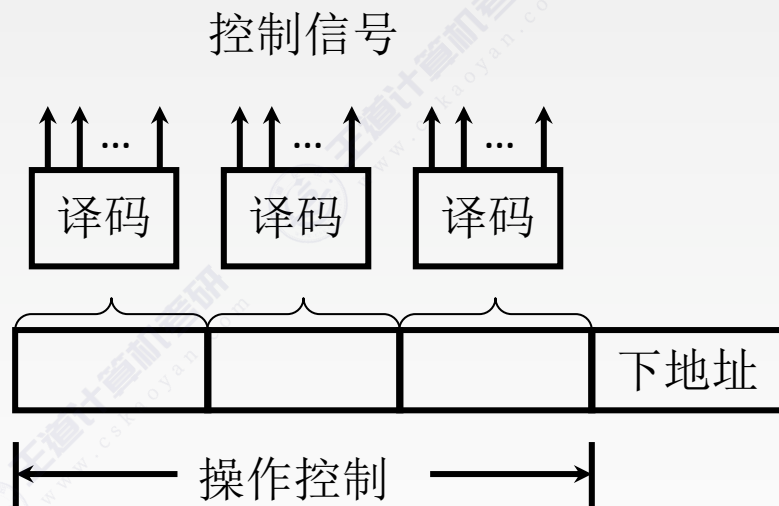
Tips: 若采用**直接编码**方式，则控制字段需要33位

微指令的编码方式

(2) 字段直接编码方式

将微指令的控制字段分成若干“段”，每段经译码后发出控制信号
微命令字段分段的原则：

- ① 互斥性微命令分在同一段内，相容性微命令分在不同段内。
- ② 每个小段中包含的信息位不能太多，否则将增加译码线路的复杂性和译码时间。
- ③ 一般每个小段还要留出一个状态，表示本字段不发出任何微命令。因此，当某字段的长度为3位时，最多只能表示7个互斥的微命令，通常用000表示不操作。

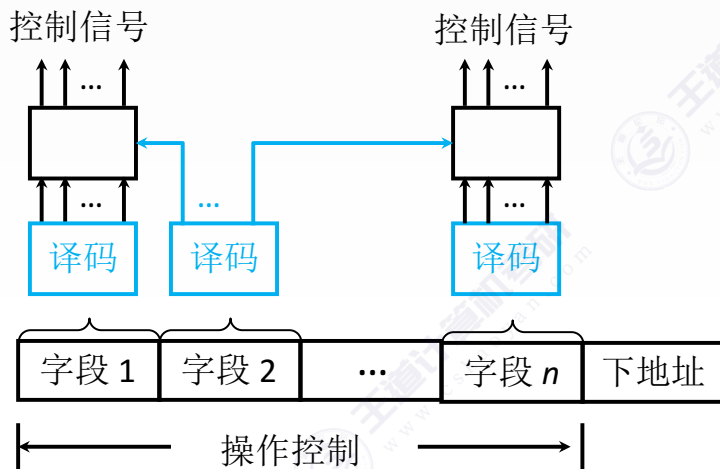


优点：可以缩短微指令字长。

缺点：要通过译码电路后再发出微命令，因此比直接编码方式慢。

(3) 字段间接编码方式

一个字段的某些微命令需由另一个字段中的某些微命令来解释，由于不是靠字段直接译码发出的微命令，故称为字段间接编码，又称隐式编码。

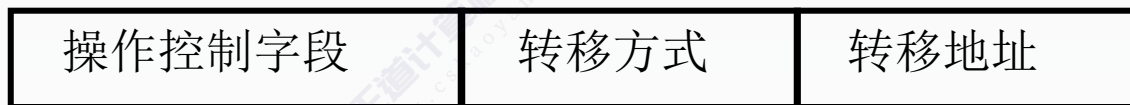


优点：可进一步缩短微指令字长。

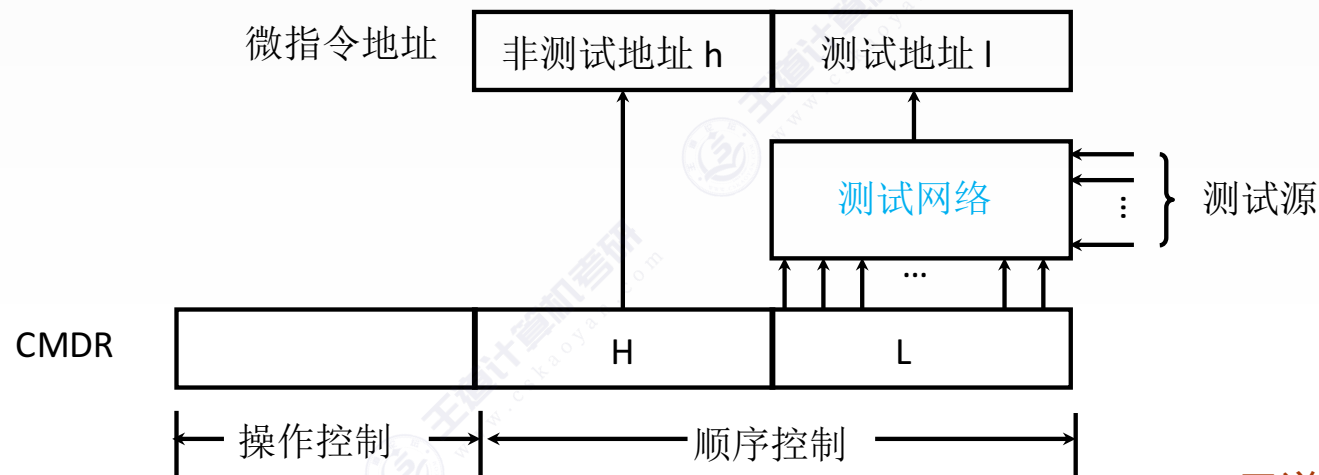
缺点：削弱了微指令的并行控制能力，故通常作为字段直接编码方式的一种辅助手段。

微指令的地址形成方式

1. 微指令的 **下地址字段** 指出 微指令格式中设置一个下地址字段，由微指令的下地址字段直接指出后继微指令的地址，这种方式又称为**断定方式**。
2. 根据机器指令的 **操作码** 形成 当机器指令取至指令寄存器后，微指令的地址由操作码经微地址形成部件形成。
3. 增量**计数器法** $(CMAR) + 1 \longrightarrow CMAR$
4. 分支转移 转移方式：指明判别条件；转移地址：指明转移成功后的去向。



5. 通过测试网络



微指令的地址形成方式

1. 微指令的 **下地址字段** 指出 微指令格式中设置一个下地址字段，由微指令的下地址字段直接指出后继微指令的地址，这种方式又称为**断定方式**。
2. 根据机器指令的 **操作码** 形成 当机器指令取至指令寄存器后，微指令的地址由操作码经微地址形成部件形成。
3. 增量**计数器法** $(CMAR) + 1 \longrightarrow CMAR$
4. 分支转移 转移方式：指明判别条件；转移地址：指明转移成功后的去向。

操作控制字段	转移方式	转移地址
--------	------	------

5. 通过测试网络
6. 由硬件产生微程序入口地址

第一条微指令地址 由专门 **硬件** 产生（用专门的硬件记录取指周期微程序首地址）

中断周期 由 **硬件** 产生 **中断周期微程序首地址**（用专门的硬件记录）

例题-断定方式

1. 微指令的 **下地址字段** 指出 微指令格式中设置一个下地址字段，由微指令的下地址字段直接指出后继微指令的地址，这种方式又称为**断定方式**。

某计算机采用微程序控制器，共有32条指令，公共的取指令微程序包含2条微指令，各指令对应的微程序平均由4条微指令组成，采用**断定法**（下地址字段法）确定下条微指令地址，则微指令中下地址字段的位数至少是多少位？

例题-断定方式

微指令
存放地
址

微指令操作码	下地址	
	1	取指周期 微程序
	2	
	3	
⋮		可以没有
转执行周期		
⋮		间址周期 微程序
转取指周期		
	14	中断周期 微程序
	15	
	0	
	17	对应 LDA 指令的 执行周期微程序
	18	
	0	
⋮		对应 STA 指令的 执行周期微程序

某计算机采用微程序控制器，共有32条指令，公共的取指令微程序包含2条微指令，各指令对应的微程序平均由4条微指令组成，采用**断定法**（下地址字段法）确定下条微指令地址，则微指令中下地址字段的位数至少是多少位？

总共需要存储多少条微指令？

$$32 \times 4 + 2 = 130 \text{ 条}$$

标注出130个不同的位置至少需要多少个二进制位？

$$2^7 = 128, 2^8 = 256$$

下地址字段的位数至少是8位

知识回顾

微指令的设计

微指令格式

水平型微指令



每条微指令能定义多个可并行的微命令

垂直型微指令



每条微指令只能定义一个微命令，由微操作码指明

混合型微指令



在垂直型微指令的基础上加上一些简单的并行操作

水平型微指令的编码方式

直接编码（直接控制）



控制码的每个 bit 对应一个微命令，微指令执行速度最快

字段直接编码



将互斥性的微命令分在同一个段内，相容的分在不同的段

每个段留出一个状态表示“不操作”

微指令操作码需要经过译码电路处理，因此执行速度更慢

字段间接编码



一个字段的微命令需要用另一个字段的微命令解释

可能需要多级译码电路处理，执行速度最慢

下一条微指令地址的形成方式

断定法（下地址法）：根据当前执行的微指令下地址找到下一条微指令

计数器法： $\mu PC + 1$ 顺序找到下一条微指令

根据指令操作码确定执行周期微程序首地址

由专门的硬件指明取指/中断周期的微程序首地址



公众号：王道在线



b站：王道计算机教育



抖音：王道计算机考研

本节内容

微程序控制单元的设计

微程序控制单元的设计

设计步骤:

1. 分析每个阶段的微操作序列
2. 写出对应机器指令的微操作命令及节拍安排
3. 确定微指令格式
4. 编写微指令码点

取指周期-硬布线控制器的节拍安排

T_0 $PC \rightarrow MAR$

T_0 $1 \rightarrow R$

T_1 $M(MAR) \rightarrow MDR$

T_1 $(PC) + 1 \rightarrow PC$

T_2 $MDR \rightarrow IR$

T_2 $OP(IR) \rightarrow ID$

取指周期-微程序控制器的节拍安排

T_0 $PC \rightarrow MAR$

T_0 $1 \rightarrow R$

T_1 $M(MAR) \rightarrow MDR$

T_1 $(PC) + 1 \rightarrow PC$

T_2 $MDR \rightarrow IR$

T_2 $OP(IR) \rightarrow \text{微地址形成部件}$

3 条微指令

微程序控制单元的设计

取指周期-硬布线控制器的节拍安排

T_0 $PC \rightarrow MAR$

T_0 $1 \rightarrow R$

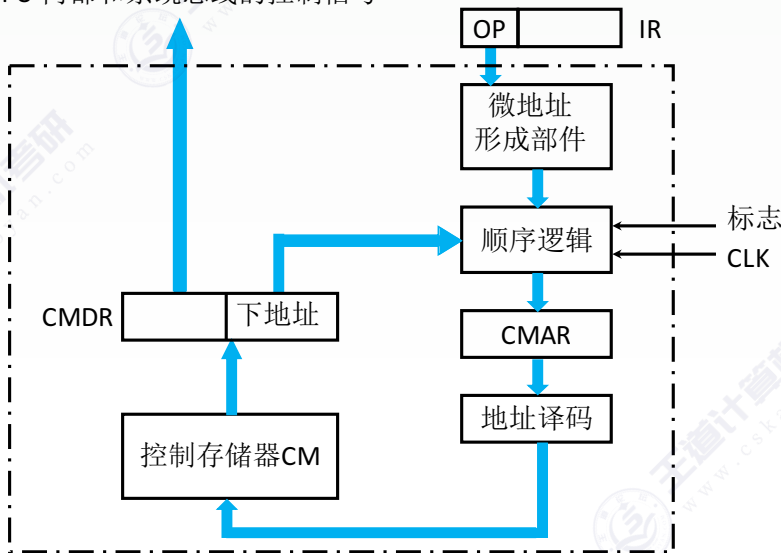
T_1 $M(MAR) \rightarrow MDR$

T_1 $(PC) + 1 \rightarrow PC$

T_2 $MDR \rightarrow IR$

T_2 $OP(IR) \rightarrow ID$

至 CPU 内部和系统总线的控制信号



取指周期-微程序控制器的节拍安排

T_0 $PC \rightarrow MAR$

T_0 $1 \rightarrow R$

T_1 $M(MAR) \rightarrow MDR$

T_1 $(PC) + 1 \rightarrow PC$

T_2 $MDR \rightarrow IR$

T_2 $OP(IR) \rightarrow$ 微地址形成部件

还需考虑 如何读出 这 3 条微指令，
以及如何转入下一个机器周期

$Ad(CMDR) \rightarrow CMAR$

$OP(IR) \rightarrow$ 微地址形成部件 $\rightarrow CMAR$

取指周期的
第一条微指
令地址由硬
件自动给出

微指令a

用微指令a的下地
址表示 b 的地址

微指令b

微指令c

用当前微指令的下地址
表示找到下一条微指令

微程序控制单元的设计

取指周期-硬布线控制器的节拍安排

T_0 $PC \rightarrow MAR$

T_0 $1 \rightarrow R$

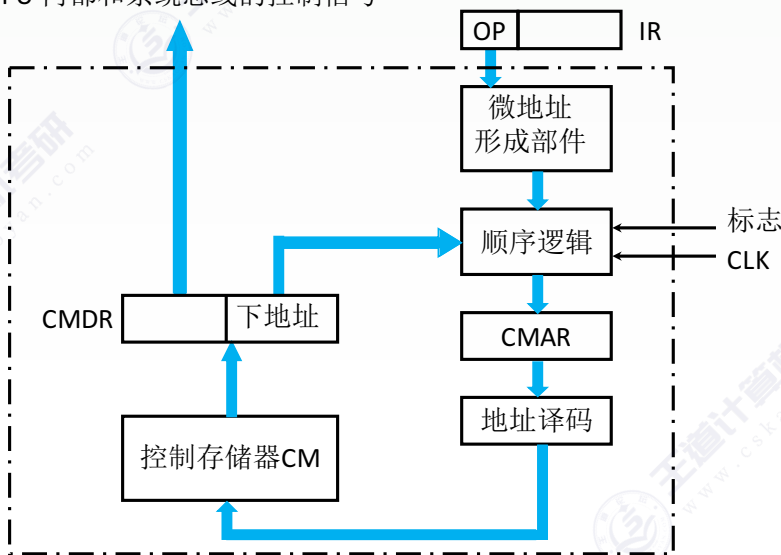
T_1 $M(MAR) \rightarrow MDR$

T_1 $(PC) + 1 \rightarrow PC$

T_2 $MDR \rightarrow IR$

T_2 $OP(IR) \rightarrow ID$

至 CPU 内部和系统总线的控制信号



取指周期-微程序控制器的节拍安排

T_0 $PC \rightarrow MAR$

T_0 $1 \rightarrow R$

微指令a

T_1 $Ad(CMDR) \rightarrow CMAR$

需要用 T_1 节拍确定下一条微指令的地址

T_2 $M(MAR) \rightarrow MDR$

T_2 $(PC) + 1 \rightarrow PC$

微指令b

T_3 $Ad(CMDR) \rightarrow CMAR$

需要用 T_3 节拍确定下一条微指令的地址

T_4 $MDR \rightarrow IR$

T_4 $OP(IR) \rightarrow$ 微地址形成部件

微指令c

T_5 微地址形成部件 $\rightarrow$ CMAR

根据指令操作码确定其执行周期微指令序列的首地址

显然，微程序控制器的速度比硬布线控制器更慢

微程序控制单元的设计

取指周期-硬布线控制器的节拍安排

T_0 $PC \rightarrow MAR$

T_0 $1 \rightarrow R$

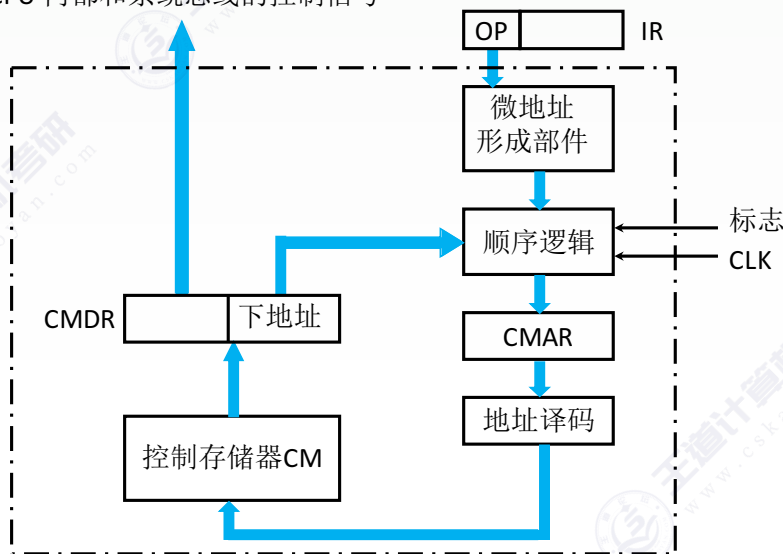
T_1 $M(MAR) \rightarrow MDR$

T_1 $(PC) + 1 \rightarrow PC$

T_2 $MDR \rightarrow IR$

T_2 $OP(IR) \rightarrow ID$

至 CPU 内部和系统总线的控制信号



取指周期-微程序控制器的节拍安排

T_0 $PC \rightarrow MAR$

T_0 $1 \rightarrow R$

微指令a

T_1 $Ad(CMDR) \rightarrow CMAR$

T_2 $M(MAR) \rightarrow MDR$

T_2 $(PC) + 1 \rightarrow PC$

微指令b

T_3 $Ad(CMDR) \rightarrow CMAR$

T_4 $MDR \rightarrow IR$

微指令c

T_5 $OP(IR) \rightarrow \text{微地址形成部件} \rightarrow CMAR$

取指周期最后一条微指令完成后，用一个特殊的微操作确定执行周期的微程序首地址

显然，微程序控制器的速度比硬布线控制器更慢

微程序控制单元的设计

设计步骤:

1. 分析每个阶段的微操作序列
2. 写出对应机器指令的微操作命令及节拍安排
 - (1) 写出每个周期所需要的微操作(参照硬布线)
 - (2) 补充微程序控制器特有的微操作:

每条微指令结束
之后都需要进行

a. 取指周期:

$Ad(CMDR) \rightarrow CMAR$

$OP(IR) \rightarrow \text{微地址形成部件} \rightarrow CMAR$

取指周期的最后一条微指令完成后, 要根据指令操作码确定其执行周期的微程序首地址

每条微指令结束
之后都需要进行

b. 执行周期:

$Ad(CMDR) \rightarrow CMAR$

3. 确定微指令格式
4. 编写微指令码点

微程序控制单元的设计

设计步骤:

1. 分析每个阶段的微操作序列
2. 写出对应机器指令的微操作命令及节拍安排
 - (1) 写出每个周期所需要的微操作(参照硬布线)
 - (2) 补充微程序控制器特有的微操作:

a. 取指周期:

$Ad(CMDR) \rightarrow CMAR$

$OP(IR) \rightarrow CMAR$

b. 执行周期:

$Ad(CMDR) \rightarrow CMAR$

3. 确定微指令格式

根据微操作个数决定采用何种编码方式, 以确定微指令的操作控制字段的位数。

根据CM中存储的微指令总数, 确定微指令的顺序控制字段的位数。

最后按操作控制字段位数和顺序控制字段位数就可确定微指令字长。

4. 编写微指令码点

微程序控制单元的设计

设计步骤:

1. 分析每个阶段的微操作序列
2. 写出对应机器指令的微操作命令及节拍安排
 - (1) 写出每个周期所需要的微操作(参照硬布线)
 - (2) 补充微程序控制器特有的微操作:

a. 取指周期:

$Ad (CMDR) \rightarrow CMAR$

$OP (IR) \rightarrow CMAR$

b. 执行周期:

$Ad(CMDR) \rightarrow CMAR$

3. 确定微指令格式

根据微操作个数决定采用何种编码方式, 以确定微指令的操作控制字段的位数。

根据CM中存储的微指令总数, 确定微指令的顺序控制字段的位数。

最后按操作控制字段位数和顺序控制字段位数就可确定微指令字长。

4. 编写微指令码点

根据操作控制字段每一位代表的微操作命令, 编写每一条微指令的码点。

微程序设计分类

1. 静态微程序设计和动态微程序设计

静态 微程序无需改变，采用 **ROM**

动态 通过 **改变微指令** 和 **微程序** 改变机器指令
有利于仿真，采用 **EPROM**

2. 毫微程序设计

毫微程序设计的基本概念

微程序设计 用 **微程序解释机器指令**

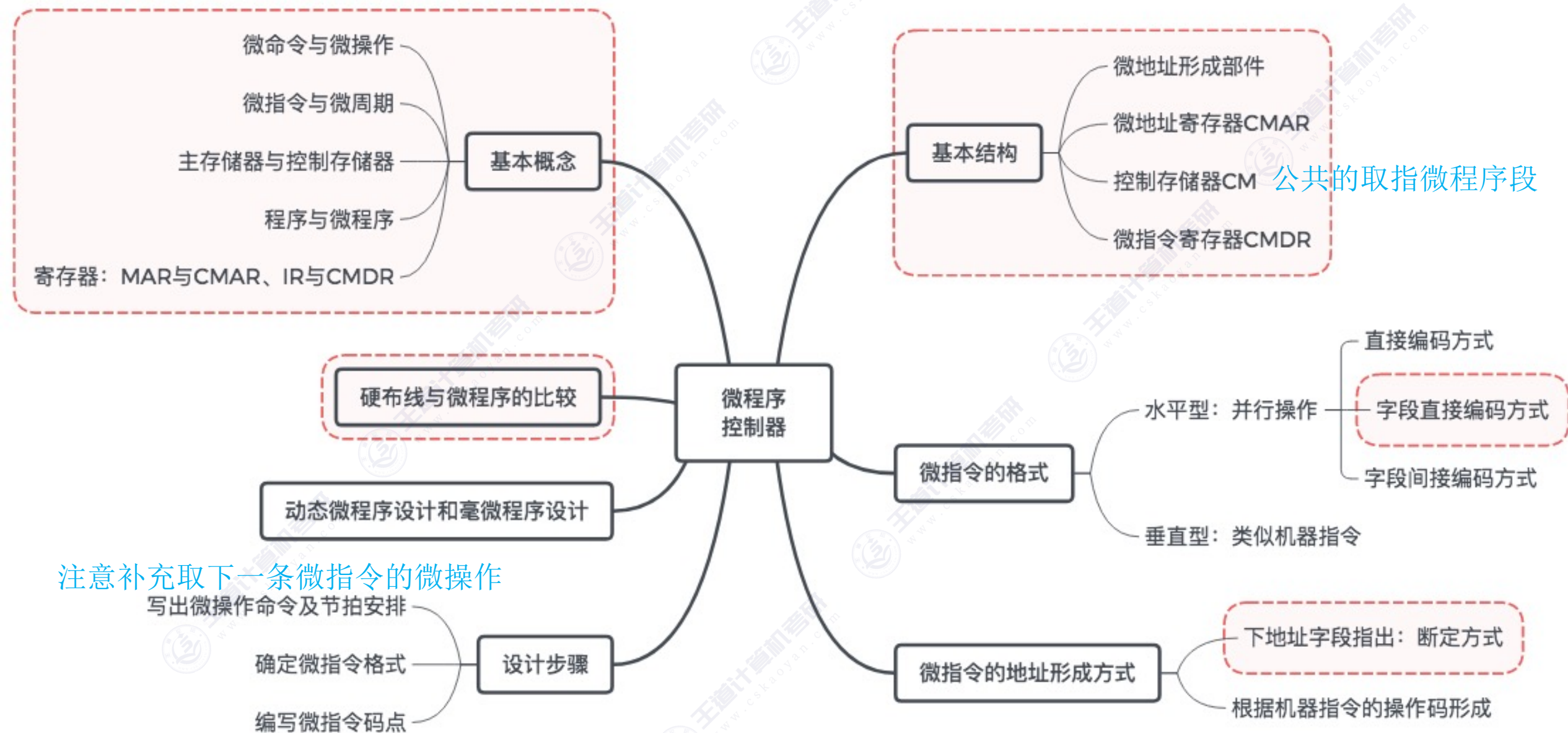
毫微程序设计 用 **毫微程序解释微程序**

毫微指令与微指令 的关系好比 **微指令与机器指令** 的关系

硬布线与微程序的比较

类 别 对比项目	微程序控制器	硬布线控制器
工作原理	微操作控制信号以微程序的形式存放在控制存储器中，执行指令时读出即可	微操作控制信号由组合逻辑电路根据当前的指令码、状态和时序，即时产生
执行速度	慢	快
规整性	较规整	烦琐、不规整
应用场合	CISC CPU	RISC CPU
易扩充性	易扩充修改	困难

微程序控制器回顾





公众号：王道在线



b站：王道计算机教育



抖音：王道计算机考研